



Trabalho Prático: Autómatos Finitos

Detetor de Partículas usando um AFD

U.C. Dados e Computação

Diana Demyanchuk dos Santos Almeida (a109793)

João Carlos Monteiro Ferreira (a110507)

Escola de Engenharia, Universidade do Minho
Campus de Gualtar, 4710-057 Braga, Portugal

17 de Maio 2026



Conteúdo

1	Introdução	3
1.1	Introdução Teórica	3
1.2	Uso no CERN	3
2	O nosso código	4
3	Conclusão	4



Resumo

Este relatório descreve a implementação em Python de um Autômato Finito Determinístico (AFD). O projeto visa aplicar os conceitos aprendidos na UC na criação de um validador de *strings*, modelado especificamente para simular o sistema de *trigger* de um detetor de partículas subatômicas, distinguindo partículas válidas de ruído físico.

1 Introdução

O presente trabalho surge no âmbito da avaliação da disciplina e tem como objetivo principal a implementação e manipulação de um autômato finito. O grupo decidiu aplicar a teoria dos autômatos a um cenário prático, o reconhecimento de assinaturas de partículas subatômicas.

1.1 Introdução Teórica

Na teoria da computação, um Autômato Finito Determinístico (AFD) é uma máquina de estados matemática que aceita ou rejeita cadeias de símbolos. Formalmente, o nosso autômato A é definido pelo quintuplo $A = (Q, \Sigma, \delta, q_0, F)$, onde:

- Q (Conjunto de Estados):

$$Q = \{q_{\text{colisao}}, q_{\text{rasto}}, q_{\text{fotao}}, q_{\text{eletrao}}, q_{\text{hadrao}}, q_{\text{muao}}, q_{\text{ruído}}\}$$

- Σ (Alfabeto):

$$\Sigma = \{T, E, H, M\}$$

- q_0 (Estado Inicial): q_{colisao}

- F (Estados Válidos): $F = \{q_{\text{fotao}}, q_{\text{eletrao}}, q_{\text{hadrao}}, q_{\text{muao}}\}$

- δ (Função de Transição): Mapeamento $\delta : Q \times \Sigma \rightarrow Q$. Qualquer transição não prevista para uma dada letra conduz ao estado de rejeição (poço) $q_{\text{ruído}}$.

1.2 Uso no CERN

No Grande Colisor de Hadrões (LHC) do CERN, ocorrem milhões de colisões de partículas por segundo. É impossível armazenar todos estes dados, pelo que o CERN utiliza sistemas de *hardware*, como FPGAs que funcionam como Autômatos Finitos. Estes sistemas, chamados de *Triggers*, leem os sensores ativados e decidem se os dados devem ser guardados ou descartados.

O nosso alfabeto (Σ) reflete as várias camadas de um detetor real: **T** (Tracker), **E** (ECAL), **H** (HCAL) e **M** (Câmara de Muões). A função de transição do nosso autômato imita isto, por exemplo, um fóton não tem carga elétrica, logo passa invisível pelo *Tracker* e para no *ECAL* (assinatura "E"). Já um eletrão deixa rasto no *Tracker* e para no *ECAL* (assinatura "TE").



2 O nosso código

A implementação da máquina de estados foi desenvolvida em Python, adotando o paradigma de programação orientada a objetos. Criou-se a classe `AutomatoDetetorParticulas`, cuja estrutura de dados central (a função δ) assentou num dicionário de dicionários (`dict`). Esta estrutura garante uma procura com complexidade temporal $O(1)$, otimizando o percurso do autômato ao ler a palavra.

A função principal, `analisar_assinatura(palavra)`, percorre a sequência de sensores através de um ciclo iterativo, saltando entre estados consoante o mapeamento do dicionário. No final da *string*, valida se o estado alcançado se encontra no conjunto F , aceitando ou rejeitando a leitura.

Além disto, o nosso trabalho também integra:

- **Geração Aleatória:** Recorrendo à biblioteca `random`, o programa permite simular N colisões gerando palavras aleatórias do alfabeto.
- **Representação Visual:** Utilizando a biblioteca `Graphviz`, processa o dicionário de transições e desenha em tempo real o diagrama de estados (colorindo a verde os estados de válidos e a vermelho o estado inválido).
- **Terminal Interativo:** Um ciclo contínuo permite a validação manual de palavras submetidas via teclado.

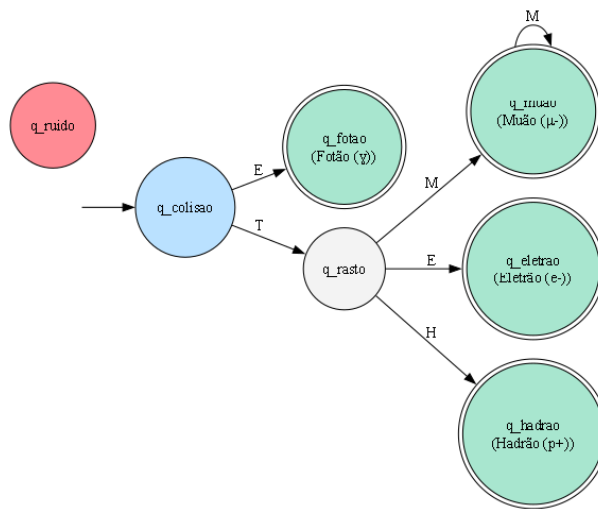


Figura 1: Diagrama do AFD

3 Conclusão

O desenvolvimento deste projeto permitiu cimentar os conceitos teóricos lecionados, comprovando que estruturas simples como os autômatos finitos possuem um vasto poder de processamento em cenários de classificação e validação. A utilização de dicionários em Python provou ser a ferramenta ideal para emular a matemática dos grafos e das funções de transição. O código completo podem ser executado no Google Colab.



Referências Bibliográficas

- [1] CMS Collaboration, CERN. *How CMS detects particles*. <https://cms.cern/news/how-cms-detects-particles>
- [2] W3Schools. *Python Tutorial*. <https://www.w3schools.com/python/>
- [3] TutorialsPoint. *Automata Theory Tutorial*. https://www.tutorialspoint.com/automata_theory/index.htm